

Task 3 — Standards Study: Sync-Status Fields → Live Sources

Mapping detection features to IEEE-1588, O-RAN YANG, and linuxptp/synce4l live sources

Four Live Sources

1. pmc (PTP Management Client)

Part of linuxptp v4.0 (current Ubuntu package); queries a running ptp4l daemon over a local Unix Domain Socket or network; returns IEEE-1588 standard management datasets. Primary source for PTP clock state once ptp4l is running on testbed. Call: `pmc -u -b 0 'GET TIME_STATUS_NP'`

2. synce4l (Intel SyncE Implementation)

Implements ITU-T G.8264 (SyncE/EEC); processes ESMC frames; exposes live query API over its own AF_UNIX socket with a binary TLV protocol. Built separately from linuxptp core (separate GitHub repo), not in Ubuntu linuxptp package by default. Requires: make synce4l against linuxptp source tree. Note: vendor-specific SyncE stacks (e.g., Renesas synced) have different API shapes — confirm post-hardware-selection.

3. O-RAN M-plane YANG (o-ran-sync.yang)

Standardized, vendor-neutral schema that an O-RU exposes to management systems over NETCONF. Not a tool you run directly; rather, the target shape your detection features should map to if claiming O-RAN compliance. Useful for reporting even if prototype polls pmc/synce4l sockets directly. Source: O-RAN Software Community GitHub (o-ran-sc/sim/o1-interface); stable version at <https://specifications.o-ran.org> (search O-RAN.WG4.MP-YANGs).

4. IEEE-1588 / ITU-T G.8275.1

Underlying standards that define the data sets pmc literally queries: defaultDS, currentDS, parentDS, timePropertiesDS, portDS. G.8275.1 adds telecom-profile refinements around BMCA and clock-quality thresholds. Field names in pmc output map directly to IEEE-1588 struct fields.

Feature → Source Mapping (Part 1: Core PTP Status)

Detection Feature	What It Tells You	pmc (linuxptp)	IEEE-1588
Master/GM offset	Current time error vs. master (ns)	TIME_STATUS_NP.master_offset	currentDS.offsetFromMaster
Mean path delay	Network delay estimate (round-trip)	CURRENT_DATA_SET.meanPathDelay	currentDS.meanPathDelay
GM present/identity	Is there a trusted GM; who is it	TIME_STATUS_NP (gmPresent, gmIdentity)	parentDS.grandmasterIdentity
GM clock quality	GM trustworthiness (class/accuracy/variance)	PARENT_DATA_SET.grandmasterClockQuality	parentDS.grandmasterClockQuality
GM priority1/priority2	BMCA tie-break inputs; spoof tell	PARENT_DATA_SET (priority1, priority2)	parentDS (priority1, priority2)
Steps removed	Hops to GM; re-parenting hint	CURRENT_DATA_SET.stepsRemoved	currentDS.stepsRemoved
Port/sync state	Port health: MASTER/SLAVE/PASSIVE/FAULTY	PORT_DATA_SET.portState (9 IEEE states)	portDS.portState

Feature → Source Mapping (Part 2: Attack Detection & SyncE)

Detection Feature	What It Tells You	pmc (linuxptp)	sync4l
Sync/FollowUp mismatch	Direct replay-attack counter (critical)	PORT_SERVICE_STATS_NP (sync_mismatch, followup_mismatch)	—
Timeout counters	Missing expected messages; DoS clue	PORT_SERVICE_STATS_NP (sync_timeout, announce_timeout, delay_timeout, master_sync_timeout)	—
Per-message-type counters	Message rate features; DoS detection	PORT_STATS_NP (rxMsgType[], txMsgType[])	—
SyncE quality level (QL)	Frequency-source trust level (G.8264)	—	MSG_GET_QL / MSG_GET_EXT_QL over AF_UNIX API
SyncE lock state	Is frequency source (EEC) locked	—	get_eec_state_cmd (LOCKED / HOLDOVER / FREERUN / INVALID)

Feature → Source Mapping (Part 3: O-RAN YANG & Upstream References)

Detection Feature	O-RAN M-plane YANG	Notes
Core PTP status	sync-status/* (time-error, sync-state); ptp-status/* (clock-identity, lock-state)	Maps to pmc TIME_STATUS_NP family
PTP sources/GM	sources/clock-identity, sources/priority1, sources/priority2, sources/clock-class, sources/clock-accuracy, sources/offset-scaled-log-variance, sources/steps-removed, sources/time-source	Rich YANG container; vendor can report multiple candidate GMs with their attributes
SyncE status	sync-e-status/lock-state; sync-e-status/sources/quality-level	G.8264-aligned; quality-level enum includes SSM levels (PRS/STU/ST2/ST3E etc.)
GNSS status (critical for attack #5)	gnss-status/gnss-sync-status (SYNCHRONIZED / ACQUIRING-SYNC / ANTENNA-DISCONNECTED / BOOTING / ANTENNA-SHORT-CIRCUIT); gnss-data/satellites-tracked, gnss-data/gnss-rx-time-error	Standards-defined tamper indicators (ANTENNA-DISCONNECTED / ANTENNA-SHORT-CIRCUIT) are your smoking gun for GNSS attack detection
PTP config	ptp-config/domain-number, ptp-config/g-8275-1-config, ptp-config/ptp-profile,	Define which profile/domain the O-RU is running; must match grandmaster

Source Confidence & Verification

pmc (high confidence): Field names are exact struct names from linuxptp v4.0 source (ds.h for standard IEEE datasets, tlv.h for linuxptp-specific NP extensions). Once ptp4l is running, live call format: `pmc -u -b 0 'GET TIME_STATUS_NP' | pmc -u -b 0 'GET CURRENT_DATA_SET'` etc.

synce4l (high confidence, but separate build): Field names sourced from `synce_external_api.h` (Intel repo). AF_UNIX socket protocol differs from pmc — your live-feed wiring needs two separate client implementations. Build: clone `linuxptp` + run `make synce4l` in same tree.

O-RAN M-plane YANG (high confidence on field names, verify version): Names copied directly from `o-ran-sync.yang` (O-RAN-SC GitHub, verified stable version). YANG evolves; confirm against latest zip at <https://specifications.o-ran.org> before final citation. Current: O-RAN.WG4.MP-YANGs (stable release).

Note on implementation: Your research prototype does NOT need to run a full NETCONF server to use these features. Polling `pmc/synce4l` sockets directly (Task 5) is simpler and what your testbed will actually do. YANG column matters for reporting and standards-alignment in your paper — reviewers familiar with O-RAN will recognize the mapping.

Real Implementation Reference

StarlingX (github.com/starlingx) ships open-source implementations of O-RAN O2 notification APIs, including REST endpoints for PTP sync status (e.g., `GET /ocloudNotifications/v2/.../sync/ptp-status/lock-state/CurrentState`). Not core M-plane NETCONF, but real running code if YANG specs feel abstract. Useful code-reading resource if your team wants to ground the standards in executable reference implementations.

Verification Checklist for Your Team

- Install `linuxptp v4.0` on Task 5 testbed machine; test pmc queries against a running `ptp4l` instance before running live tests
- Clone `synce4l` separately; confirm AF_UNIX socket protocol matches `synce_external_api.h` from repo
- Read `o-ran-sync.yang` directly from O-RAN-SC GitHub; confirm enum values for `gnss-sync-status` and `ptp-status/sources/state` match latest published version
- Once hardware is selected (Task 4), check NIC vendor's `synce4l` fork or proprietary SyncE daemon; confirm API compatibility before deployment
- Cross-reference `pmc/synce4l` live feeds with your digital-twin simulation to ensure feature ingestion logic is identical